



Learn. From the world's best.

Linear Algebra Examples and Functions

A hand from the right side of the frame points towards a glowing brain icon. The brain is composed of circuit-like patterns and is surrounded by a dense network of glowing blue lines that resemble a circuit board or neural network. The background is dark blue with a subtle pattern of light blue dots and lines.

The Importance of Linear Algebra in ML and Data Science



1

Foundation for ML algorithms

2

Powers matrix operations & data transformations

3

Enables efficient data processing

Matrices in ML

Importance of Matrices & Transformations



Represent data (features, images, text embeddings)



Used for scaling, rotation, and projection



Speeds up model training & inference

Matrix Computations in ML Models

How ML Models Rely on Matrices

- **Neural Networks:** Weight matrices process inputs
- **Dimensionality Reduction:** PCA uses eigenvalues/eigenvectors
- **Linear Regression:** Normal Equation applies matrix inversion
- **Optimisation:** Gradient computation via matrix calculus



Matrix Operations and Applications in ML

Where It's Used?

- **Computer Vision:** Image processing
- **NLP:** Word embeddings, similarity
- **Recommendation Systems:** Matrix factorisation
- **Deep Learning:** Activations & weight updates

Essential Computations

- Fast data processing and transformations
- Matrix multiplication, inner products, solving equations

Neural Network Forward Propagation

Example: Forward Pass

Multiplication: Inputs $\times$ weights

Input features are multiplied by weight matrices at each layer

Formula:

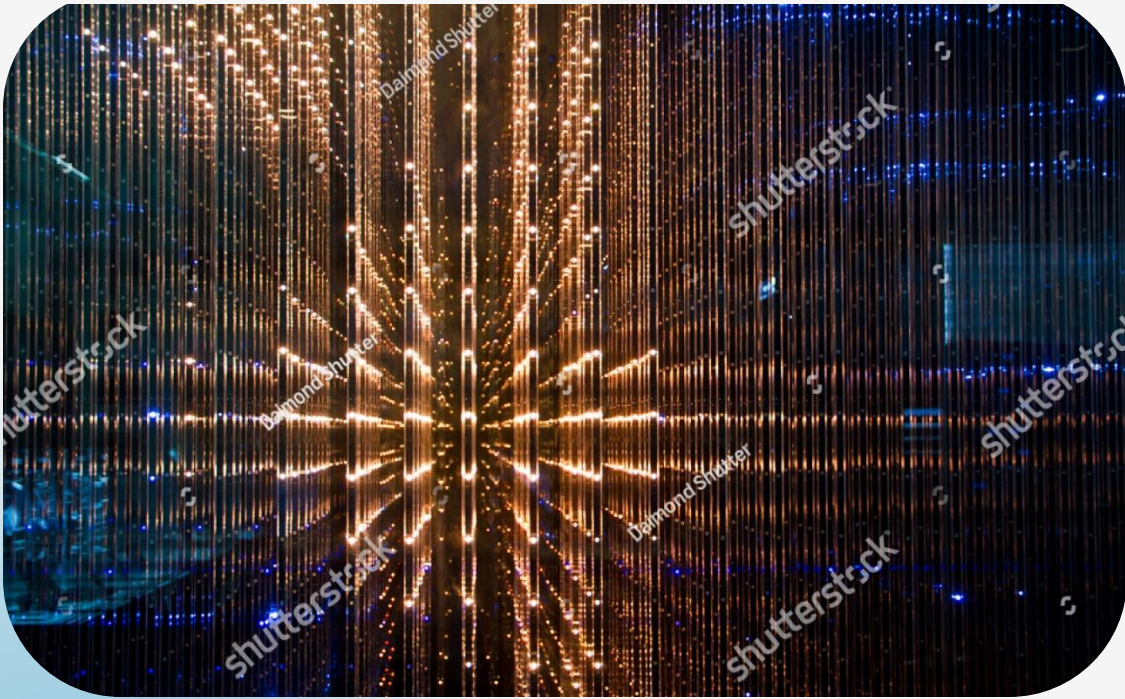
$$\mathbf{Z} = \mathbf{W} \mathbf{X} + \mathbf{B}$$

Where $\mathbf{W}$ is the weight matrix, $\mathbf{X}$ is the input, and $\mathbf{B}$ is the bias term

Efficiency: Enables fast predictions

Matrix Multiplication and Batch Processing

How It Enhances Efficiency



1

Parallel Processing: Handles multiple inputs at once

2

Optimised Computation: Faster ML training

3

Example: Batch training in deep learning

Inner Product in ML

Measuring Similarity

Definition: Dot product of two vectors

Formula:

$$[\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i]$$

Cosine similarity in NLP & recommendations

Cosine Similarity in Recommendation Systems

Similarity Measurement

Measures similarity between users/items based on preferences

Formula:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

Application: Used in collaborative filtering and content-based recommendation

Solving Equations and Matrix Decompositions in ML

Finding Model Parameters



- ML models solve for unknowns using matrix equations
- **Example:** Normal Equation in Linear Regression
- Linear regression finds the best-fit line for a dataset
- **Normal Equation:**
$$\theta = (X^T X)^{-1} X^T y$$
- **Efficiency:** Removes the need for iterative gradient descent in some cases

Eigenvalues, Eigenvectors and SVD

Key Concepts for ML

Eigenvalues, eigenvectors, and SVD are essential concepts in linear algebra

Used in ML for dimensionality reduction, feature extraction, and efficient computation

Power Principal Component Analysis (PCA), Image compression, and NLP



Enhancing Data Analysis with Dimensionality Reduction

Benefits of Dimensionality Reduction:

- Reduces computation
- Removes noise
- Helps in feature extraction
- Enhances visualisation of high-dimensional data

Eigenvectors and Eigenvalues in Feature Extraction:

- Used in PCA to identify the principal components
- Captures the directions of maximum variance in the dataset

PCA with Eigenvectors and Eigenvalues

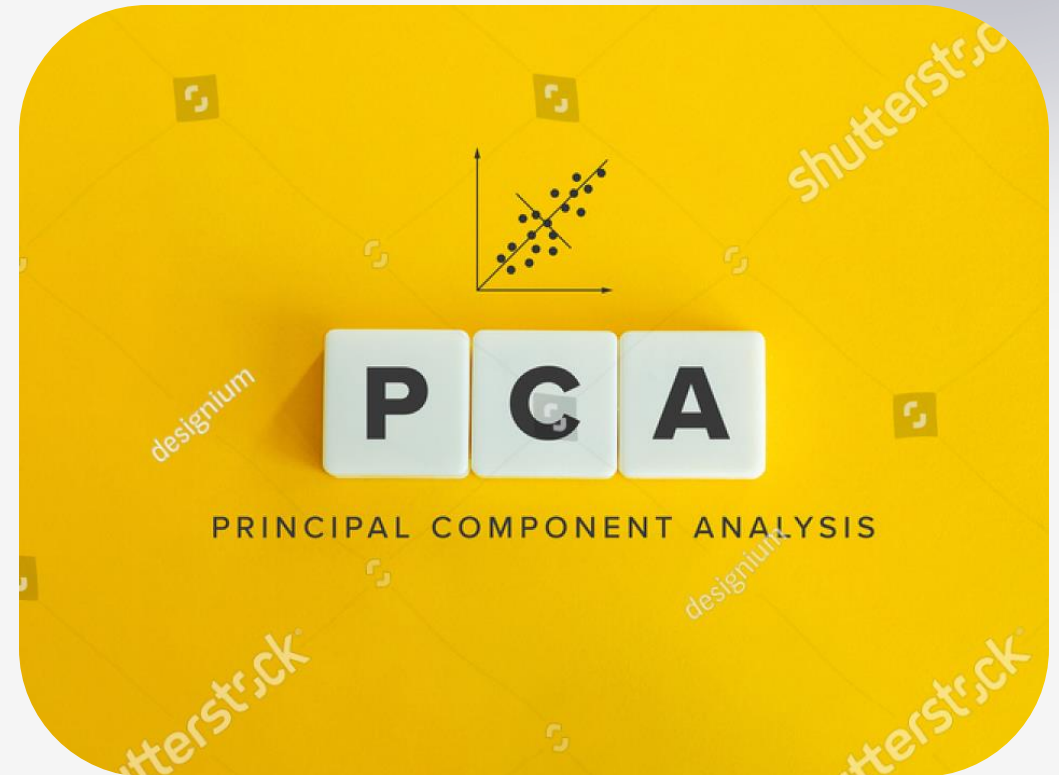
PCA reduces high-dimensional data using eigenvectors and eigenvalues

Dimensionality Reduction simplifies data while preserving key patterns

Feature Extraction enhances interpretability and removes noise

Applications include image compression, NLP, and pattern recognition

Use: Predicting numerical outcomes from features



SVD in ML Applications

Definition:

$$[A = U \Sigma V^T]$$

Where,

(U): Left singular vectors

(Σ): Diagonal matrix with singular values

(V^T): Right singular vectors

Uses: Reduces complexity, removes noise, extracts key features, and improves data compression for efficient processing

SVD in Image Compression and Latent Semantic Analysis in NLP

Image Compression Using SVD

- Images are stored as matrices
- SVD extracts key features while removing noise
- Retaining only the top singular values reduces size while preserving structure
- **Application:** Saves storage with minimal loss of quality

Latent Semantic Analysis (LSA) in NLP

- Converts text into a term-document matrix
- SVD extracts hidden topics
- Helps with text summarisation and retrieval
- **Application:** Search engines, recommendation systems, and document clustering

Why Decompositions Matter in ML

Speeding Up Computations

- Efficient factorisation methods improve processing speed
- Reduces the complexity of large datasets

Regularisation & Interpretability

- Helps prevent overfitting by simplifying models
- Enhances the understanding of data structures



ML Models in Matrix Form

Linear Regression

- Many ML models use matrix representations for efficient computation
- Examples include linear regression, logistic regression, and neural networks

Linear Regression:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Why it Matters?

- Direct solution without iterative optimisation
- Computationally expensive for large datasets

Application:

- Predicting numerical outcomes
- from feature matrices

ML Models in Matrix Form

Logistic Regression

Sigmoid Function Representation:

$$[\sigma(z) = 1 / \{1 + e^{-z}\}] \text{ where } (z = Xw)$$

Gradient Descent in Matrix Notation:

$$[w = w - \alpha \{\partial L / \partial w\}]$$

where (L) is the loss function

Application:

- Binary classification problems

Neural Networks and Matrix Computations

Forward Pass Representation:

$$[Z = WX + b]$$

- Weighted sum of inputs
- Activation functions applied element-wise

Backpropagation and Gradient Updates:

- Uses derivatives and matrix operations to update weights
- Efficient computation through vectorised implementations

Google PageRank Algorithm

Purpose:

- Ranks web pages based on importance using eigenvectors

Mathematical Foundation:

- The web is represented as a directed graph
- A link matrix (adjacency matrix) represents page connections
- Eigenvector computation determines the importance of pages

Key Equation:

$$[\mathbf{PR} = \beta (\mathbf{M} \mathbf{PR}) + (1 - \beta) \mathbf{e}]$$

where:

- $(\mathbf{PR})$ is the PageRank vector
- $(\mathbf{M})$ is the link matrix
- (β) is the damping factor
- $(\mathbf{e})$ represents random jumps

Applications:

- Used by search engines to rank webpages effectively

Eigenvectors in Web Ranking

Powering Search Engines



- Eigenvectors identify influential webpages
- Approximates dominant eigenvectors
- Captures the stationary distribution of the link matrix
- Helps search engines rank relevant pages

Recommendation System

Collaborative Filtering

- **Purpose:** Predicts user preferences based on past behaviour
- **Matrix factorisation:** Decomposes the user-item interaction matrix and identifies latent factors influencing recommendations
- **Common Techniques:**
 - **Singular Value Decomposition (SVD):** Finds optimal latent factors
 - **Alternating Least Squares (ALS):** Optimises recommendations iteratively

Applications:

- Used in Netflix, Amazon, Spotify for better recommendations

ML Models in Matrix Form

Linear Regression

Problem: Sparse user-item matrix with missing values

Solution: Matrix Factorisation finds user preferences and item traits

Equation:

$$[R \approx U \Sigma V^T]$$

Where,

(R) is the user-item matrix

(U) represents users latent features

(Σ) contains singular values (importance of features)

(V) represents item features

Impact:

- Improves recommendation accuracy and personalises content

Facial Recognition Using Eigenfaces



- Recognises faces by reducing dimensionality using PCA
- Identifies variations using Eigenfaces

How It Works:

- Converts facial images into greyscale and computes the covariance matrix
- Uses eigenvectors (Eigenfaces) to capture variations and represent new faces

Application:

- Security, authentication, and social media tagging

PCA for Facial Recognition

Dimensionality Reduction for Efficient Processing

Problem: High-dimensional images are costly to process

Solution: PCA reduces dimensions while retaining key features

Equation:

$$[X' = X W]$$

Where,

(X) is the original face image matrix

(W) contains the top eigenvectors (Eigenfaces)

(X') is the reduced representation

Impact:

- Enables fast and accurate facial recognition while reducing storage and processing requirements